

Getting Started

This notebook goes over how to use the LLM class in LangChain.

The LLM class is a class designed for interfacing with LLMs. There are lots of LLM providers (OpenAI, Cohere, Hugging Face, etc) - this class is designed to provide a standard interface for all of them. In this part of the documentation, we will focus on generic LLM functionality. For details on working with a specific LLM wrapper, please see the examples in the [How-To section](#).

For this notebook, we will work with an OpenAI LLM wrapper, although the functionalities highlighted are generic for all LLM types.

```
from langchain.llms import OpenAI
```

```
llm = OpenAI(model_name="text-ada-001", n=2, best_of=2)
```

Generate Text: The most basic functionality an LLM has is just the ability to call it, passing in a string and getting back a string.

```
llm("Tell me a joke")
```

```
'\n\nWhy did the chicken cross the road?\n\nTo get to the other side.'
```

Generate: More broadly, you can call it with a list of inputs, getting back a more complete response than just the text. This complete response includes things like multiple top responses, as well as LLM provider specific information

```
llm_result = llm.generate(["Tell me a joke", "Tell me a poem"]*15)
```

```
len(llm_result.generations)
```

```
llm_result.generations[0]
```

```
[Generation(text='\n\nWhy did the chicken cross the road?\n\nTo get to the other side!'),  
 Generation(text='\n\nWhy did the chicken cross the road?\n\nTo get to the other side.')] ]
```

```
llm_result.generations[-1]
```

```
[Generation(text="\n\nWhat if love never speech\n\nWhat if love never ended\n\nWhat if love was only a feeling\n\nI'll never know this love\n\nIt's not a feeling\n\nBut it's what we have for each other\n\nWe just know that love is something strong\n\nAnd we can't help but be happy\n\nWe just feel what love is for us\n\nAnd we love each other with all our heart\n\nWe just don't know how\n\nHow it will go\n\nBut we know that love is something strong\n\nAnd we'll always have each other\n\nIn our lives."),  
 Generation(text='\n\nOnce upon a time\n\nThere was a love so pure and true\n\nIt lasted for centuries\n\nAnd never became stale or dry\n\nIt was moving and alive\n\nAnd the heart of the love-ick\n\nIs still beating strong and true.')] ]
```

You can also access provider specific information that is returned. This information is NOT standardized across providers.

```
llm_result.llm_output
```

```
{'token_usage': {'completion_tokens': 3903,  
 'total_tokens': 4023,  
 'prompt_tokens': 120}}
```

Number of Tokens: You can also estimate how many tokens a piece of text will be in that model. This is useful because models have a context length (and cost more for more tokens), which means you need to be aware of how long the text you are passing in is.

Notice that by default the tokens are estimated using a HuggingFace tokenizer.

```
llm.get_num_tokens("what a joke")
```